software

MDPI

Article

Characterizing Agile Software Development: Insights from a Data-Driven Approach Using Large-Scale Public Repositories

Carlos Moreno Martínez [1], Jesús Gallego Carracedo [2] and Jaime Sánchez Gallego [3,*]

[1] Computing and Technology Department, School of Architecture, Engineering and Design, Villaviciosa Campus, Universidad Europea de Madrid, Villaviciosa de Odón, 28670 Madrid, Spain; carlos.moreno@universidadeuropea.es
[2] Independent Researcher, Villaviciosa de Odón, 28670 Madrid, Spain; jesus_gc_93@hotmail.com
[3] Department of Industrial Engineering, Higher Polytechnic School, Campus Madrid—Princesa, Antonio de Nebrija University, 2815 Madrid, Spain
* Correspondence: jsancheg@nebrija.es

Abstract: This study investigates the prevalence and impact of Agile practices by leveraging metadata from thousands of public GitHub repositories through a novel data-driven methodology. To facilitate this analysis, we developed the AgileScore index, a metric designed to identify and evaluate patterns, characteristics, performance and community engagement in Agile-oriented projects. This approach enables comprehensive, large-scale comparisons between Agile methodologies and traditional development practices within digital environments. Our findings reveal a significant annual growth of 16% in the adoption of Agile practices and validate the AgileScore index as a systematic tool for assessing Agile methodologies across diverse development contexts. Furthermore, this study introduces innovative analytical tools for researchers in software project management, software engineering and related fields, providing a foundation for future work in areas such as cost estimation and hybrid project management. These insights contribute to a deeper understanding of Agile's role in fostering collaboration and adaptability in dynamic digital ecosystems.

Keywords: Agile methodologies; hybrid methodologies; software development; GitHub repositories; data-driven analysis; project classification; open-source projects

check for updates

Academic Editor: Andrea Pinna

Received: 8 February 2025
Revised: 24 April 2025
Accepted: 21 May 2025
Published: 24 May 2025

Citation: Moreno Martínez, C.; Gallego Carracedo, J.; Sánchez Gallego, J. Characterizing Agile Software Development: Insights from a Data-Driven Approach Using Large-Scale Public Repositories. *Software* **2025**, *4*, 13. https://doi.org/10.3390/software4020013

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In the digital economy, effective project management is essential for organizational success. Agile methodologies have become increasingly relevant due to their ability to adapt to changing requirements and foster collaboration among development teams compared to traditional development models. However, it is essential to comprehend the contextual realities of software projects and develop robust mechanisms to assess their productivity across diverse environments. Although Agile methodologies are widely supported in theory and practice, conducting large-scale comparative studies remains challenging due to the lack of identification tools capable of distinguishing traditional projects from Agile ones, as well as the absence of prior large-scale data and examples for both types of projects. In this context, this study establishes a central research area related to project management research: establishing which methodologies are more relevant in terms of popularity and community engagement, with the related challenge of the large-scale identification of applied development models in software projects, in order to enable comparative assessments through the evaluation of real-world cases. To address this

limitation and provide answers to the research question, this study explores thousands of software projects, analyzing their metadata from public repositories (GitHub), applying data mining and analysis techniques.

Agile project management has been extensively adopted due to its emphasis on flexibility, iterative development and continuous customer collaboration, which help teams to respond more effectively to changing requirements and market demands. These characteristics make Agile particularly suitable for dynamic and complex software development environments [1]. Specific project management frameworks and best practice collection guidelines have been published to support the growing interest in and practical implementation of Agile methods [2].

Therefore, this study focuses on three key research objectives:

RO1. To develop a study model for analyzing software projects in digital repositories, enabling large-scale comparisons between Agile and traditional software projects. This model will focus on identifying the degree of agility by evaluating specific attributes associated with development methodologies. Projects with consistently high and low levels of agility will be selected to ensure the robustness of the analysis and facilitate comparison, serving as inputs to the next objective. In this context, large-scale means conducting the analysis across a wide and diverse set of software projects, allowing for the identification of generalizable patterns and trends in Agile and traditional development practices based on publicly available data.

RO2. To characterize, by applying the study model, the status of software projects in public repositories. Specific goals are used to evaluate project dynamics in terms of developer community interest and collaboration, as well as to understand if there is a shift in the fundamental coverage of development models of Agile vs. traditional approaches in open-source software development or even the emergence and prevalence of hybrid approaches. In this context, project dynamics refers to organizational aspects and the temporal evolution of development activity within a project, including patterns of commits and release cadence.

RO3. To create a replicable process for the large-scale analysis of projects in public digital repositories. As a final aim, this study wants to provide a robust and generalizable framework for researchers to investigate the evolution of software development practices within dynamic digital ecosystems.

By applying the defined model, we have observed that projects using Agile methodologies tend to receive higher valuations from the developer community, as evidenced by factors such as a higher number of watchers, suggesting a greater degree of success compared to projects based on other methodologies. Furthermore, this application supports the observation of a clear upward trend in the adoption of Agile methodologies in recent years. These findings not only highlight the increasing adoption of Agile practices but also raise important questions about their potential impact on key project outcomes, such as software quality, development speed and overall project success.

It is important to emphasize that our study does not aim to improve the Agile process itself or identify which specific process features contribute to greater project management efficiency. Rather, the goal is to provide a method for identifying software projects that follow specific approaches (agile, classic or hybrid), thereby facilitating further research into the factors that influence project success and efficiency in Agile environments.

This paper summarizes the relevant research literature and presents the research background in Section 2. Section 3 outlines the study design and methodology, along with the steps taken to conduct the research. Sections 4 and 5 present this study's findings and the discussion, followed by conclusions and future work in Section 6.

2. Literature Review

The Agile approach is widely utilized in software development [3] and represents a significant departure from traditional methods of establishing relationships between business and development teams [4]. The trend in adopting Agile approaches in project management is also impacting or clashing with high maturity levels [5]. Agile-oriented project management is a methodology that emphasizes iterative development, continuous feedback and adaptability to changing requirements. Unlike traditional project management approaches, such as the Waterfall model, Agile frameworks (e.g., Scrum, Kanban and XP) prioritize customer collaboration, incremental deliveries and self-organizing teams. The key distinctions lie in Agile's flexibility, responsiveness to change and focus on delivering value through short development cycles. In contrast, traditional methodologies often follow a sequential, plan-driven approach with rigid phases. Specific aspects of Agile techniques, such as the impact of Agile-related practices on project outcomes and team productivity, have been studied [6,7], as well as different methods for effort estimation in Agile-oriented development [8]. Recent research explores the trend of customizing project management models by hybridizing development processes, moving beyond the traditional dichotomy between Agile and conventional models ([9,10]). Kirpitsas and Pachidis [11] provide a comprehensive list of hybrid methods and mention the work of Glass [12] as a key driver in the advance of hybrid software development methods.

Mining software repositories, such as GitHub, provides a rich source of data for project analysis. GitHub is the most widely used platform for version control during project development both on a personal and professional level. As of the date of preparation of this study, it contains more than 420 million repositories of actual projects, of which more than 284 million are publicly accessible. It stands as one of the largest and most comprehensive repositories of public software development projects [13].

Numerous studies have utilized GitHub as a primary data source in project management and software engineering research [14]. Project dynamics can be analyzed based on contributions from developers and software development approaches identified in Q&A forums [15] or interest from the development community like the number of watchers [16]. Research has also paid attention to behavioral patterns through the analysis of specific attributes present in GitHub metadata [17], developer behavior [18] and social coding and collaboration [19]. Metadata can be used to understand development history [20], and several previous works apply Artificial Intelligence (AI) and machine learning techniques to study software cost and effort estimation ([21–23]) or perform tracking and labeling [24].

Several prior studies have contributed to the field by constructing curated datasets tailored to specific aspects of software development. The TAWOS dataset [25], comprising over 500,000 issues from 44 Agile open-source projects, facilitates research into various aspects of Agile development. The PHANTOM method [26] employs time-series clustering to identify software projects in a computationally efficient manner. Both works provide valuable methodological contributions: TAWOS offers a large, labeled dataset that supports the empirical analysis of Agile projects, and PHANTOM introduces an unsupervised approach to identify software repositories based on Git logs, focusing on filtering "engineered software projects". In addition, PHANTOM provides a comprehensive analysis of prior repository filtering approaches, offering insights on the criteria and challenges involved in distinguishing projects within large-scale ecosystems, as well as the technical challenges of accessing GitHub data via API or the constraints of existing datasets such as GHTorrent. However, both approaches have limitations: TAWOS focuses on a fixed set of Agile projects, which may not be suitable for capturing the diversity of development models across the broader ecosystem, while PHANTOM's methodology, centered on activity patterns, does

not explicitly distinguish between Agile and other development methodologies, limiting its applicability in comparative studies of development processes.

In summary, there has been limited focus on large-scale comparisons of traditional development models with Agile or hybrid development approaches for real-world cases. While the current body of research frequently leverages open-source repositories such as GitHub, the identification of development models across large datasets remains a significant challenge. The rise of platforms such as GitHub has enabled unprecedented opportunities for empirical analysis of software engineering practices through accessible and extensive metadata. Yet, despite the availability of large-scale datasets and the growing interest in mining repositories for Agile-related insights, there remains a notable gap in data-driven methodologies capable of systematically identifying and differentiating Agile projects from those following traditional or hybrid development models. This limitation restricts the potential for broad comparative evaluations and hinders a more nuanced understanding of how different development approaches impact project outcomes. Addressing this gap opens a promising research avenue for the creation of robust, metadata-based indicators to detect Agile practices in real-world settings, laying the groundwork for scalable assessments, performance comparisons and evidence-based process improvements in software engineering.

3. Materials and Methods

The design of the research process incorporated elements of the action research approach [27] to address "real-life practical problem situations" while maintaining a focus on the context of actual projects, thereby ensuring the validity of the results. Also, this study was inspired by the approach proposed by Hevner et al. [28], with Information Systems research being "at the confluence of people, organizations, and technology" when an IT artefact in an organizational context is studied from a behavioral science perspective.

The analysis process employed in this study was based on using data analysis techniques to provide a quantitative approach, which was essential given the nature of the subject matter [29]. This study followed the methodological framework discussed by Mariscal et al. [30], inspired by stages from the CRISP-DM model [31,32], a widely used method in the data lifecycle [33]. The analysis based on the CRISP-DM model provided a structured sequence for the data mining process. The alignment between the CRISP-DM model phases and the stages applied in this study was as follows: (a) The "Business Understanding" phase corresponded to stage 1 of this study, which aimed to define the selection criteria of the data and repositories aligned with the research objectives, as well as to identify relevant limitations of metadata in GitHub. (b) The "Data Understanding" and "Data Preparation" phases aligned with stage 2, which included identifying and selecting data sources, assessing data quality, generating initial insights and choosing a representative sample. The analysis of the attributes allowed for insight into the semantics and the potential contribution of each variable to the subsequent model. (c) The "Modeling" phase corresponded to stage 3 of this study, aimed at defining the AgileScore index and describing the process for characterizing Agile and non-Agile projects using machine learning techniques, as well as at highlighting model explainability through attribute relevance. (d) The "Evaluation" phase matched stage 4, where insights from previous stages were used to conduct visual analyses and cross-reference findings with other data to validate the results. (e) The "Deployment" phase, typically associated with product development, was excluded from this study. Instead, it was replaced by the practical application of the model to the dataset, aimed at analyzing key characteristics of Agile projects, as presented in the results section.

The methodological process of this research was based on linearly progressive stages but included various iterations and feedback loops within certain stages. In Stage 2, during data collection from repositories, a general dataset was used and processed to identify the optimal variables for this study. Once these variables were processed and filtered, it may have become apparent that additional data were needed to better align the research with its objectives. This may have led to a repetition of the data search and extraction phase using other sources. Another example of iteration occurs in stage 3 during the manual labeling of the sample. Based on the results of this process, parameters were adjusted to ensure a similar number of records were labeled with high and low AgileScore (e.g., stdDTRelease). This was carried out to avoid sample imbalance that could bias the classification model toward the predominant label. Similarly, the parameters of the classification model itself were fine-tuned based on its output in order to achieve a more balanced distribution of classifications when applying the trained model to the corresponding sample.

This structured approach ensured a comprehensive analysis, aligning this study's methodology with well-established data mining processes, thereby enhancing the robustness and reliability of the findings (Figure 1).

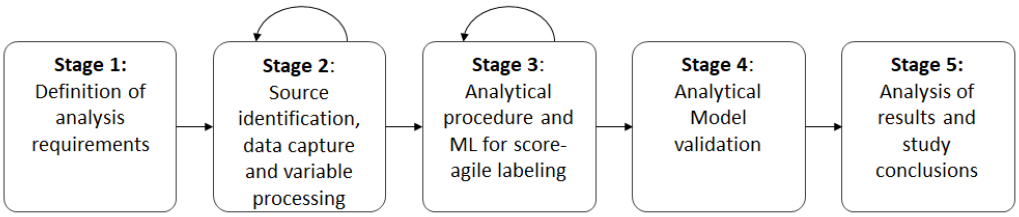

Figure 1. Research and data analysis process.

3.1. Stage 1: Definition of Analysis Requirements

The research objectives required the use of data from actual projects. Consequently, the initial step in the methodology involved identifying data repositories that provided information aligned with this study's objectives. In this context, "adequate" data referred to meeting several specific characteristics. First, the data had to fall within this study's scope in terms of the project's type; therefore, only data from projects focused on creating software products were included, excluding data from other types of repositories. Second, the data had to originate from actual projects that have demonstrably involved the delivery of a product. These projects had to be diverse regarding product size, duration, project age, development stage and the associated individuals or organizations.

Additionally, the selected projects were preferably sourced from an environment with a robust community of users and developers. This would ensure the active involvement of various participants at different project stages, including roles such as developers and observers.

The data repository requirements, as determined by the research objectives, are summarized as follows:

(a) The repositories had to contain data from software product development projects;
(b) The data had to originate from actual projects which have generated product;
(c) There would preferably be no a priori limitation regarding whether the project owner was a commercial entity or an individual;
(d) The data had to be public and sourced from a development community that operates under open-source principles [34];
(e) The data would preferably include information reflecting characteristics typical of Agile development models (e.g., frequency of product delivery), which would allow for the identification of common patterns in projects with Agile development approaches.

(f) The data had to be accessible, either through direct access (API—Application Programming Interface) or via a platform that hosts publicly accessible data repositories.

There was a dependency and limitation regarding project success metrics, as GitHub does not provide direct indicators of functional requirements' fulfillment. If a specific goal for project success had to be defined, indirect measures had to be used, such as community engagement (e.g., number of watchers), collaboration patterns (e.g., contributor activity) and temporal development dynamics (e.g., commit frequency, release cadence). However, it is important to note that the aim of this study was not to assess project success per se, but rather to enable the identification and differentiation of Agile projects from those following traditional or hybrid development models.

3.2. Stage 2: Source Identification, Data Capture and Variable Processing

A data science-driven process was undertaken to inform the construction of the study model, in alignment with the research objectives. The process began with the identification of three public data sources—Kaggle, GitHub's REST API and Google BigQuery—that satisfied the criteria defined in Stage 1. These repositories were selected for their complementary coverage of relevant project characteristics. An exploratory analysis of these sources was then conducted to assess their structure, content and analytical potential. This analysis revealed gaps in certain variables within individual sources, thereby necessitating the use of multiple datasets to ensure comprehensive coverage of the analysis criteria. Notably, GitHub's REST API was not used as the sole data source due to well-documented limitations related to data completeness, project representativeness and API constraints [35].

Building on the insights derived from this exploration, the next step focused on the identification and selection of attributes aligned with this study's objectives, particularly those indicative of agility levels. These attributes formed the analytical foundation for enabling large-scale comparisons between Agile and traditional software development practices, as outlined in research goals RO1 and RO2, and supported the development of a replicable research framework in line with RO3. Based on the selected attributes and the information obtained from the three data sources, a unified analytical dataset was constructed to support the subsequent stages of this study. The outline of the process is described below in chronological order of work:

(a) The "GitHub Public Repository Metadata" dataset available in JSON format on the Kaggle collaborative platform (https://www.kaggle.com/datasets/pelmers/github-repository-metadata-with-5-stars/data (accessed on 24 September 2023)). It has general metadata information from more than 3 million projects with 5 or more 'stars'. This first dataset was the basis for identifying study projects and the basis for which the first characterization and filtering analysis actions were carried out.

(b) Mining repository metadata through GitHub's Rest API obtained in JSON format. This data source was used to complete the overall dataset, adding the metadata corresponding to project releases (not included among Kaggle's variables). These metadata were essential for understanding the nature of projects in terms of their development model.

(c) The "github_repos" dataset available in the "Bigquery-public-data" public dataset in Google Big query (https://cloud.google.com/blog/topics/public-datasets/github-on-bigquery-analyze-all-the-open-source-code?hl=en (accessed on 24 September 2023)). Here, we found a table with the metadata of the commits of all the public repositories of GitHub. "commit", according to the GitHub glossary, is a change in one of the files in the repository made by anyone who collaborates on the project. This information (not included among Kaggle's variables) was considered important for reflecting the behavior of the developer community in projects, relevant to under-

standing the dynamics of development. Table 1 presents the number of items pulled from each of these data sources.

Table 1. Characterization of datasets (sources and main characteristics).

	Kaggle	API Mined Dataset	BigQuery
Num. of documents	3,274,587	84,970	1,180,160
Num. of repositories	3,274,587	6195	6195
Num. of attributes	25	35	8
Main contribution	Repositories metadata	Project releases data	Commit data

A NoSQL database was used to process these metadata. Specifically, a MongoDB server was deployed in an AWS cluster to enable it to work in a distributed computing environment, which allowed data transformations to be carried out quickly and efficiently. Power BI was used exclusively as a visualization tool for validating the analytical model and comparing the resulting variables, without taking part in the modeling or classification stages (Figure 2).

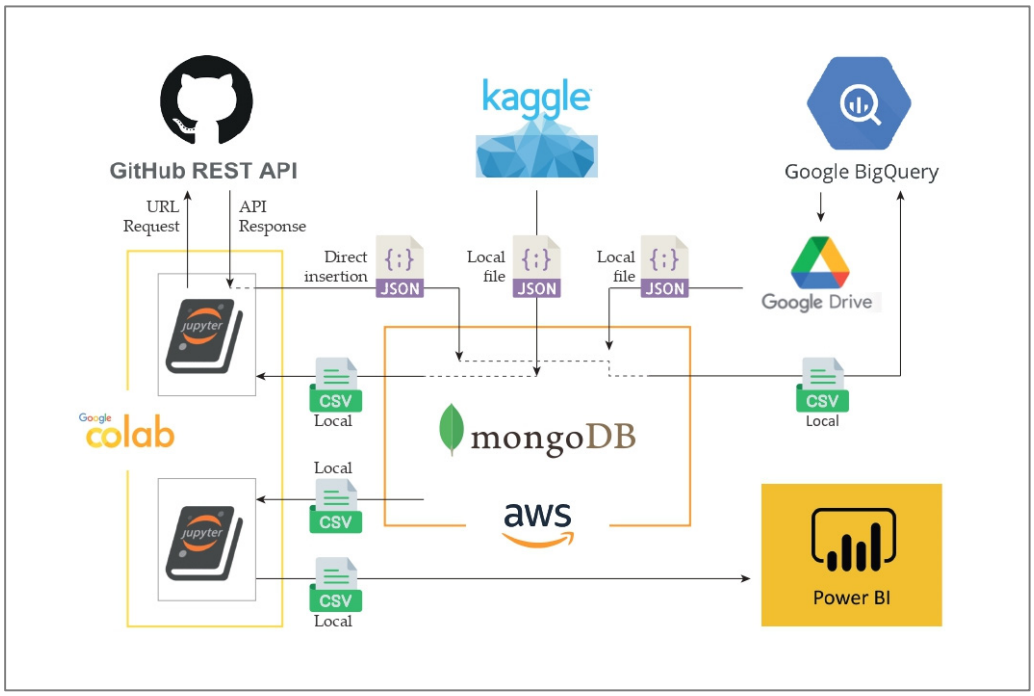

Figure 2. Architecture and components related to analysis process. Note. The specification of the architectural technical components is beyond the scope of this study. Nevertheless, this image is included as a general guideline to facilitate the reproduction of the data analysis infrastructure in similar architectures and to support the explanation of the analytical process.

To ensure that the sample consists of comparable projects, the analysis process began with the Kaggle dataset in JSON format, filtering the repositories based on three attributes and the following criteria:

(a) 'isArchived': This standard variable in GitHub repository metadata indicates projects that have concluded and have been marked as such by their developers. Those repositories that have a value of 'True' were selected to ensure that project data were used in a steady state that would not undergo modifications after the analysis process.

(b) 'diskUsageKb': This standard variable in GitHub repository metadata indicates the size of the repository on disk (measured in KB). The values of this variable in the origi-

nal dataset have an asymmetric distribution (lognormal), with great variability. It contains outliers with very large values compared to the rest of the values (μ = 26,748.72, σ = ±421,503; median: 552; min: 0; Max: 107 × 10^6; variance σ2 = 1.78 × 1011). Only repositories that have a value between 1000 and 100,000 KB in this variable were selected to ensure that atypical, immature or poorly developed projects (the typical "Hello World") were excluded. The original data distribution is shown in Figure 3 with the Probability Density Function (PDF) of the fitted lognormal distribution. A logarithmic scale was used to better visualize the data and the fitted distribution.

(c) 'description': This standard variable in GitHub repository metadata includes a description of the repository in text format. It is used to filter those projects related to software development, as there are repositories on GitHub that are not associated with software code and products. The filtering procedure involved searching this field for the presence of 24 predefined keywords commonly associated with code development. These keywords were identified through a review of the first 100 records in the dataset and included terms such as 'source', 'program', 'app', 'display', 'API', 'library', among others.

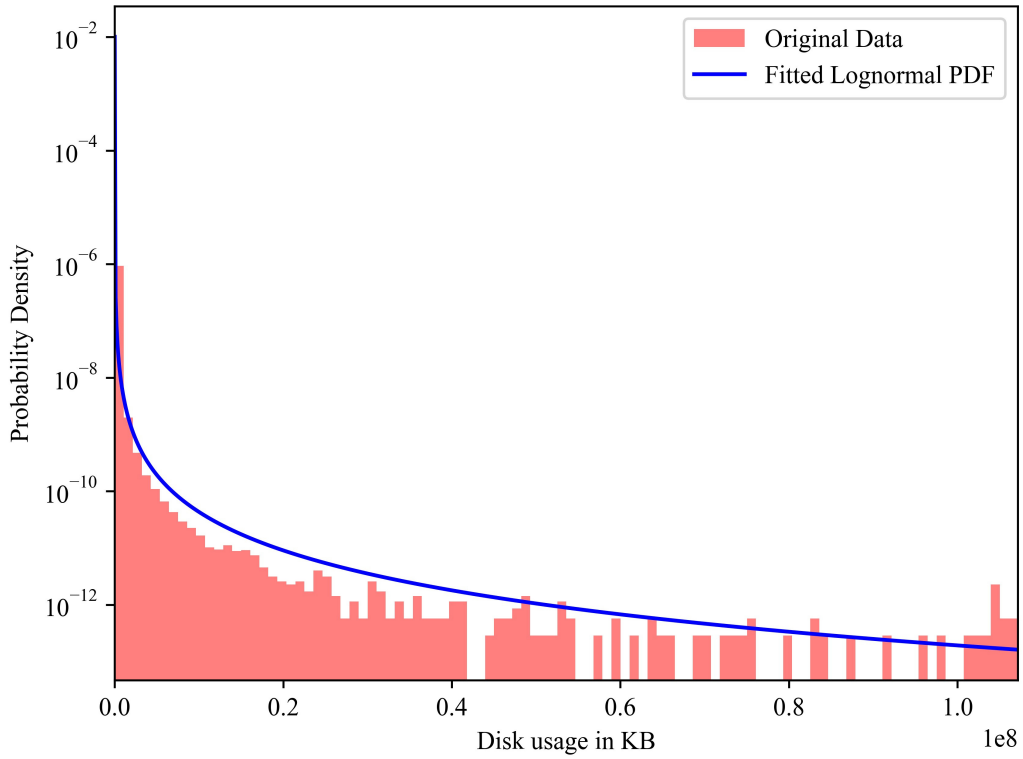

Figure 3. Distribution of GitHub repositories by diskUsageKb.

Once the sample was defined by the specified parameters, a CSV file was generated containing the identifiers of each filtered repository. This list was then used to feed a Python notebook in Google Colab (release 20 August 2024), which performed data mining through the GitHub REST API (version 2022-11-28) to retrieve release metadata for each repository (as this information was not available in any of the other datasets used). For each query made using the identifiers, the API returned a response in the form of multiple JSON documents. Each document contained the metadata of a specific release from the queried repository. These metadata constituted the second dataset and were directly inserted into the AWS URL where MongoDB (v7.0) was deployed.

The next step involved extracting the identifiers of the mined repositories and using them to retrieve commit data from the third dataset in Google BigQuery. This process was

performed using an SQL query to filter the commit data for these repositories and to select the eight most relevant variables for the analytical process. The extracted information was saved as a JSON file in Google Drive (release 11 July 2024). This file was then downloaded and uploaded to the working database. With the release information in the database, the sample was further narrowed as follows:

(d) 'tag_name': This field stores the name of each published version of the project. These metadata were used to filter projects that follow a standardized versioning nomenclature, characterized by a three-digit numeric code separated by periods (e.g., '1.0.0'). This selection ensured that the analyzed projects follow established version control practices.

After refining the sample and gathering all the necessary data, a dataset comprising 1154 documents was obtained. The metadata of these projects were processed and structured in an appropriate way for subsequent analysis. Among the most relevant transformations carried out in MongoDB were the counting of the number of languages, releases and commits; the calculation of the average time elapsed between publications of both commits and releases; and their respective standard deviations.

In the second step of the process, attributes were selected to align with this study's research objectives. Project metadata were distilled into 23 variables that form the analytical foundation for identifying Agile characteristics and capturing the most relevant aspects of project dynamics. The selected attributes contributed in different ways: some enhanced the semantic understanding of software development model characterization, others served basic repository filtering functions and some provided general contextual information. A summary of the selected variables is presented in Table 2, indicating for each attribute its name, description, role in the analysis (semantic—S, filtering—F, or general—G), data type and source dataset.

The result was a well-defined key-value data structure for each repository, enabling all the information to be organized in a structured manner within a single CSV file. In this format, each document represented a repository within the dataset. Organizing the data as a table ensured the correct implementation of the machine learning model.

Table 2. Selection of repository metadata attributes.

Attribute Name	Description	Selection Criteria	Origin	Type
owner	Name of the repository creator	F	Kaggle	String
Forks	Number of copies made of the repository	G	Kaggle	Integer
Watchers	Number of project followers	G	Kaggle	Integer
topicCount	Number of categorization labels assigned	S—The use of labels and categories could indicate an organized and modular approach, which is characteristic of Agile methodologies, where requirements and tasks are managed iteratively.	Kaggle	Integer
diskUsageKb	Disk size of repository data	G	Kaggle	Integer
pullRequests	Number of change approval requests	S—The number of pull requests is a direct indicator of collaboration and continuous code review, key elements of Agile. A high number of pull requests suggests an iterative development approach.	Kaggle	Integer
primaryLanguage	The most widely used programming language in the repository	G	Kaggle	String
defaultBranchCommitCount	Number of commits made in the repository	G	Kaggle	Integer
license	Name of the development license used	G	Kaggle	String
assignableUserCount	Number of users responsible for changes	S—A lower or moderate number of assignable users could be characteristic of a small, self-organizing Agile team, while larger teams might be more typical of traditional methodologies.	Kaggle	Integer
codeOfConduct	Rules of conduct to be followed by participants	G	Kaggle	String
nameWithOwner	Unique repository identifier (creator/project)	F	Kaggle	String
numLanguages	Number of programming languages used	S—Agile projects often use multiple technologies and programming languages, as the adaptability and integration of new tools are Agile principles. A higher number of languages might suggest a flexible and adaptive approach.	Kaggle	Integer
year	Year of creation of the repository	G	Kaggle	Integer

Table 2. *Cont.*

Attribute Name	Description	Selection Criteria	Origin	Type
topReleaserName	Name of the user who has published the most releases	G	Rest API	String
numReleases	Number of releases released	S—The number of releases reflects the ability to deliver software frequently, a fundamental principle of Agile methodologies, which emphasize small, continuous releases.	Rest API	Integer
firstReleaseDay	Number of days until the first release	S—Agile projects tend to have quicker initial releases as part of their iterative cycle.	Rest API	float
averageDTRelease	Average number of days between one release and another	S—A short release cycle suggests an Agile approach, with frequent deliveries and constant feedback, which is a hallmark of Agile methodologies.	Rest API	float
stdDTRelease	Standard deviation of days between one release and another	S—A low standard deviation indicates a consistent release rhythm, which is common in Agile teams that maintain a predictable cadence of development.	Rest API	float
topCommitterName	Repo member who has posted the most commits	G	Big Query	String
numCommits	Number of commits made in the repository	S—A high number of commits reflects frequent and continuous development, a central principle of Agile methodologies, which promote incremental, ongoing work.	Big Query	Integer
averageDTCommit	Average number of days between one commit and another	S—A short time interval between commits indicates a consistent development rhythm, which is a key feature of Agile methodologies, promoting continuous, incremental work.	Big Query	float
stdDTCommit	Standard deviation of days between commit and commit	S—A low standard deviation between commits reflects a predictable work rhythm, which is common in Agile teams that maintain a consistent development pace.	Big Query	float

3.3. Stage 3: Analytical Procedure and Machine Learning for AgileScore Labeling

This stage aimed to define the 'AgileScore' variable based on the metadata selected in the previous stage. The score ranged from 1 to 10 and allowed for the assignment of an Agile rating to each project based on these selected attributes. This scoring system enabled a clear distinction between projects developed with Agile methodologies (high AgileScore) and those that were not (low AgileScore). AgileScore is a numerical metric that quantifies the degree of Agile methodology adoption in a given project. Formally, AgileScore is a function $S{:}P \rightarrow [1, 10]$, where P denotes the set of all projects and $S(p)$ assigns a score to each project p based on metadata attributes identified during the preceding evaluation stage (delivery frequency, variability in release cadence). The score is calibrated such that higher values (e.g., $S(p) \approx 10$) indicate a strong adherence to Agile practices, whereas lower values (e.g., $S(p) \approx 1$) suggest minimal or no Agile methodology implementation.

Following the hybridization theory proposed by M. Kuhrmann et al. [36], the high likelihood of hybrid practices across a substantial portion of software projects must be acknowledged. As a result, enforcing a strict binary classification of projects is of limited value. Instead, this study adopted a percentage-based scoring approach to evaluate each repository. This method enabled the assessment of the extent to which a project's development methodology aligned with Agile principles or, alternatively, with other methodological paradigms. The resulting scoring system provided a standardized means of differentiating between Agile and non-Agile projects. Furthermore, the use of a 1–10 scale introduced greater granularity into the supervised regression process, allowing for more precise threshold definitions for Agile project classification based on the model's root mean squared error (RMSE).

A new variable 'methodology' was created to record the project classification. The specific steps followed in this label process were the following:

(a) Data cleansing. Records with outliers in the statistical distribution of any variable were removed. Additionally, this process identified fields containing null values, which were considered during data processing in the machine learning model.

(b) Manual tagging of the 'AgileScore' variable. The identification of Agile projects was based on two key attributes: 'stdDTRelease' and 'numReleases'. The 'stdDTRelease' was used as a proxy for regularity in delivery frequency, one of the core principles of Agile development, as established in references [1,11]. This variable was used to assign an AgileScore of 10 to projects with a 'stdDTRelease' value between 0 and 20 (lowest variability in cadence) and an AgileScore of 9 to projects with a 'stdDTRelease' value between 20 and 50. These thresholds were chosen to ensure a balanced distribution of positive and negative labels. For lower values of the AgileScore, the variable 'numReleases' was used. Since frequent deliveries imply the presence of multiple releases, projects with very few releases were identified as the least agile. Consequently, an AgileScore of 1 was assigned to projects with a single release and a score of 2 to projects with two releases. Moreover, 'numReleases' was used to prevent projects with few releases from being rated as highly agile, establishing that only those with more than 10 releases can be assigned a score of 9 or 10. 'stdDTRelease' was not used for assigning the lowest AgileScore values. Through these procedures, the dataset of 365 projects was labeled, which was used to train the predictive model (Table 3).

(c) Tagging intermediate Agile scores using machine learning. Since part of the sample was already labeled, supervised machine learning techniques could be applied to automatically infer intermediate values of the 'AgileScore' for the remaining samples. A Random Forest Regressor algorithm was employed to predict this value for the 786 repositories that had not been manually labeled. To apply this technique, the

sample was divided into two groups: one labeled group, used for training and testing the model in an 80/20 split, and one unlabeled group, to which the trained model was then applied. The parameters used in the model were the default settings, and a summary of some of them is provided below:

Table 3. Repository manual labeling based on numRelease and stdDTRelease.

'AgileScore'	'numReleases'	'stdDTRelease'	Label Distribution (%) in Manual Tagging (n = 365)
1	$n = 1$	Not applicable	23%
2	$n = 2$	Not applicable	32%
9	$n >= 10$	$n <= 20$	30%
10	$n >= 10$	$20 < n <= 50$	15%

- 'n_estimators' = 100;
- 'min_samples_split' = 2;
- 'max_features' = auto.

In both the training and prediction phases, the values of the remaining numerical variables in the dataset (those not used in manual tagging) were utilized to perform either training or predictions. This approach helped to mitigate biases in the model's predictions.

Once the model was trained, a Feature Importance function was applied to identify which variables in the dataset had the greatest or least influence on the predicted outcome. This method, also known as Gini Importance [37], was selected because it provides a global view of the model by averaging the influence of each feature across all predictions, which is particularly useful for general analysis and documentation purposes. Moreover, Feature Importance is easy to interpret, computationally efficient and significantly faster to compute than more complex explainability models such as SHAP or LIME. It showed that the 'averageDTRelease' variable was the most influential in predicting AgileScore values, followed by 'stdDTCommit' and 'numCommits'. The variables 'topicCount', 'numLanguages' and 'assignableUserCount' were the least relevant in this automated process (Figure 4). Additionally, the RMSE (root mean square error) of the prediction was 2.2.

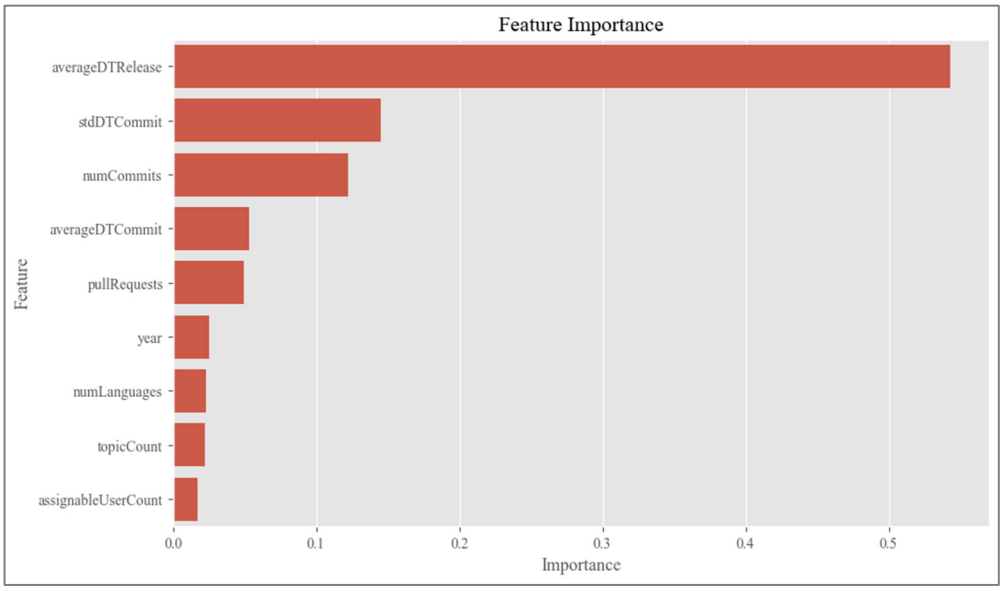

Figure 4. Feature Importance.

(d) Assignment of Agile/Non-Agile 'Methodology'. Once the 'AgileScore' was assigned to all projects, the 'methodology' variable was used to clearly classify projects as

either Agile or Non-Agile. Under an ideal, zero-error model, selecting projects with a score above the midpoint of the scale (i.e., >5 out of 10) would correctly identify those exhibiting a majority of Agile practices. However, our model's validation on held-out repositories produced a root mean squared error (RMSE) of 2.2, which implies that a midpoint threshold lay well within the model's uncertainty range and could lead to false positives. Consequently, we adopted a more conservative cutoff of ≥8 (i.e., midpoint + RMSE, rounded up) to reliably identify projects that predominantly follow Agile approaches. This conservative threshold helped to ensure that all repositories labeled as Agile truly adhered to Agile principles, even in light of the widespread hybridization of development methodologies observed in practice [36] (Figure 5).

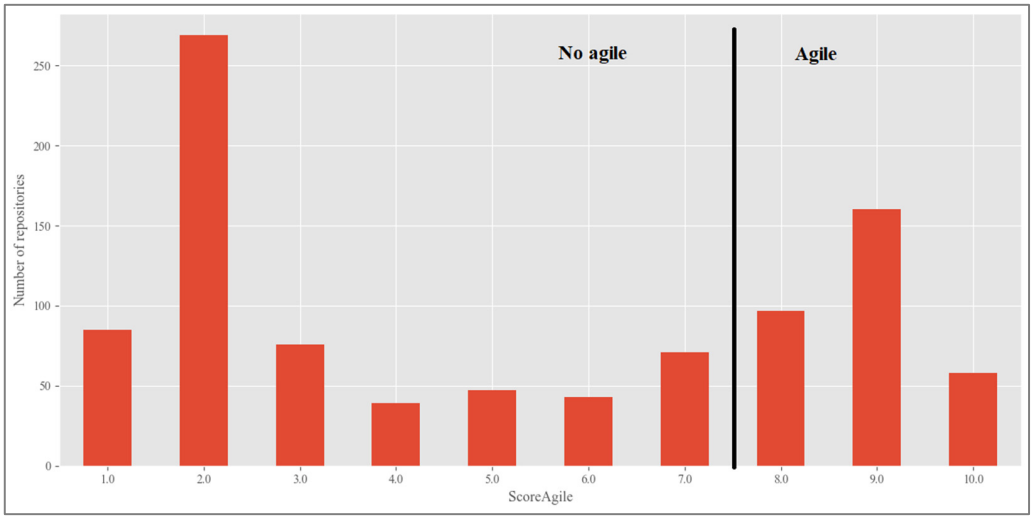

Figure 5. Repository classification based on AgileScore. The black vertical line denotes the agility threshold (≥8) separating agile from less agile projects.

3.4. Stage 4: Analytical Model Validation

The validation of the results from the previous stages is presented in two aspects. First, a trend was observed between the average delivery days (averageDTRelease) and the increase in the AgileScore, confirming one of the fundamental principles of Agile development models: the frequent delivery of product. This pattern was reflected in the machine learning model, as the intermediate values of the AgileScore follow the same trend (Figure 6).

On the other hand, projects labeled as Agile conformed to the criteria defined in the manual labeling process. These projects exhibited lower standard deviation values for the number of days between releases (stdDTRelease), indicating a consistent delivery frequency. In contrast, projects with a lower AgileScore showed greater variability in their delivery frequencies (Figure 7).

A multivariate sensitivity analysis was conducted to evaluate the combined impact of both variables. The global average shows only a slight decrease in AgileScore (−1.98%).

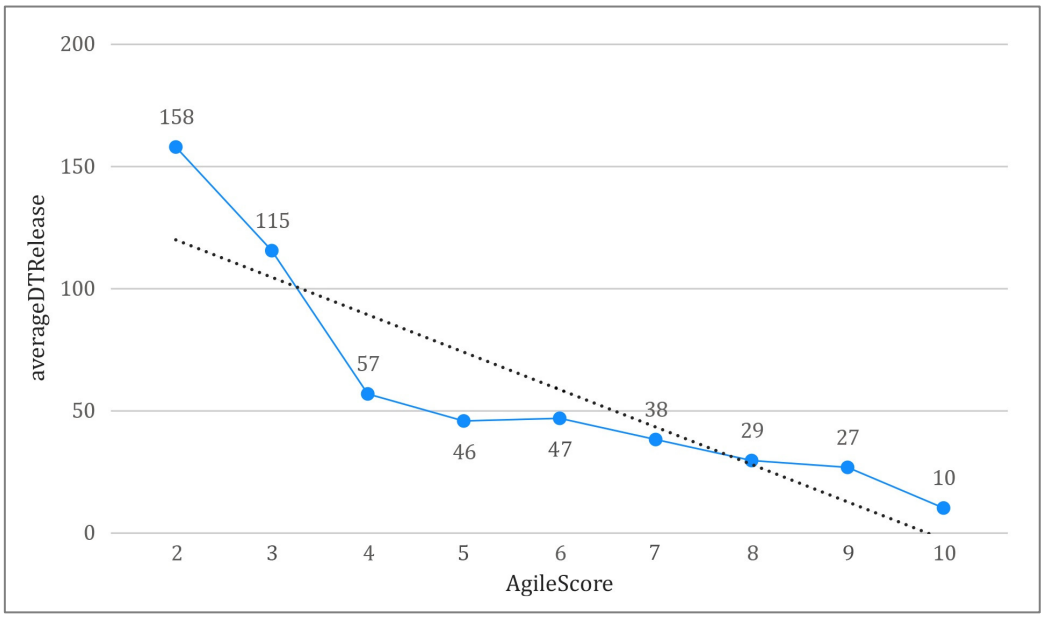

Figure 6. AgileScore vs. average date to release (averageDTRelease). A linear trendline is included to illustrate the decreasing trend in averageDTRelease as AgileScore increases.

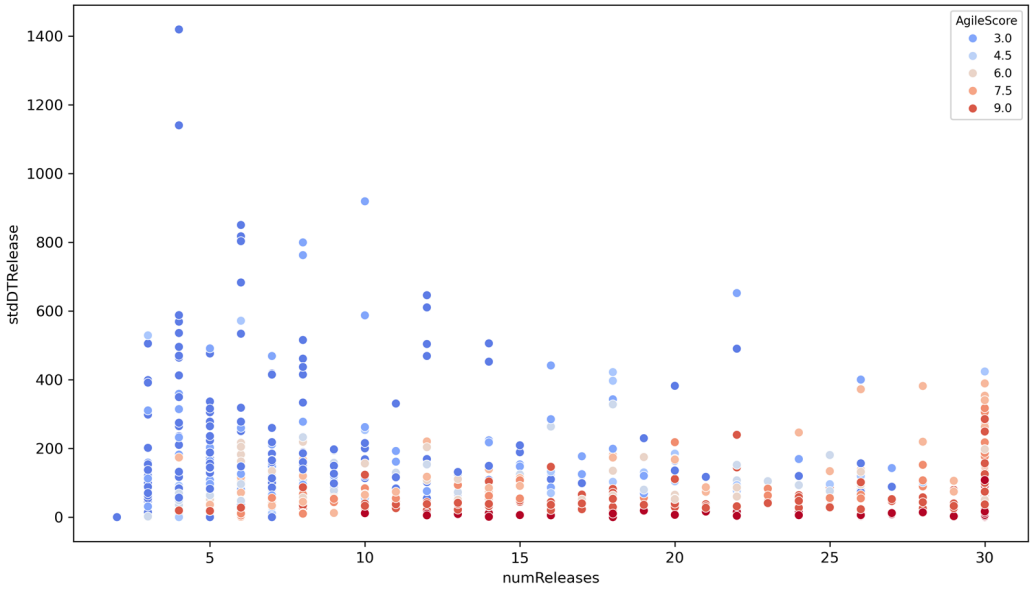

Figure 7. Variations in AgileScore related to stdDTRelease and numReleases.

4. Results

The definition of the Agile project identification model using the AgileScore index (research objective RO1) has been applied to the analysis of various project characteristics related to developer community dynamics and collaboration (research objective RO2). The data-driven analysis process defined in this study (research objective RO3) has been applied to the metadata of GitHub repositories.

The AgileScore index for identifying agile-focused projects is based on characteristic behaviors such as the frequency of product delivery and the variability in release cadence. Using AgileScore, relevant trends, patterns and conclusions were identified that contributed to the knowledge base in this field. The main findings of this study regarding Agile methodologies in software development projects through metadata mining in public GitHub repositories are as follows:

(a) In the sample of repositories to which the AgileScore index is applied (from 2011 to 2019), a growing trend in the adoption of Agile methodologies on GitHub is observed, with an average annual increase of 16% (±0.47). This trend is consistent with previous findings reported in [1]. However, while the situation identified by AgileScore across all analyzed projects reveals a clear upward trend, Agile projects do not yet constitute a prevailing majority. This is consistent with the observation made in [3], which identifies in its literature review the apparent belief that traditional processes have been entirely replaced by Agile. An anomaly in 2018 is evident but has been retained in the results to accurately reflect the actual state of the repositories. Figure 8 illustrates the evolution over time of the growth of projects marked with the AgileScore index.

(b) When applying the AgileScore index from the perspective of organizational project dynamics, such as interest among the developer community in specific repositories or projects, a pattern emerges: a higher number of watchers is observed as the AgileScore index increases (Figure 9).

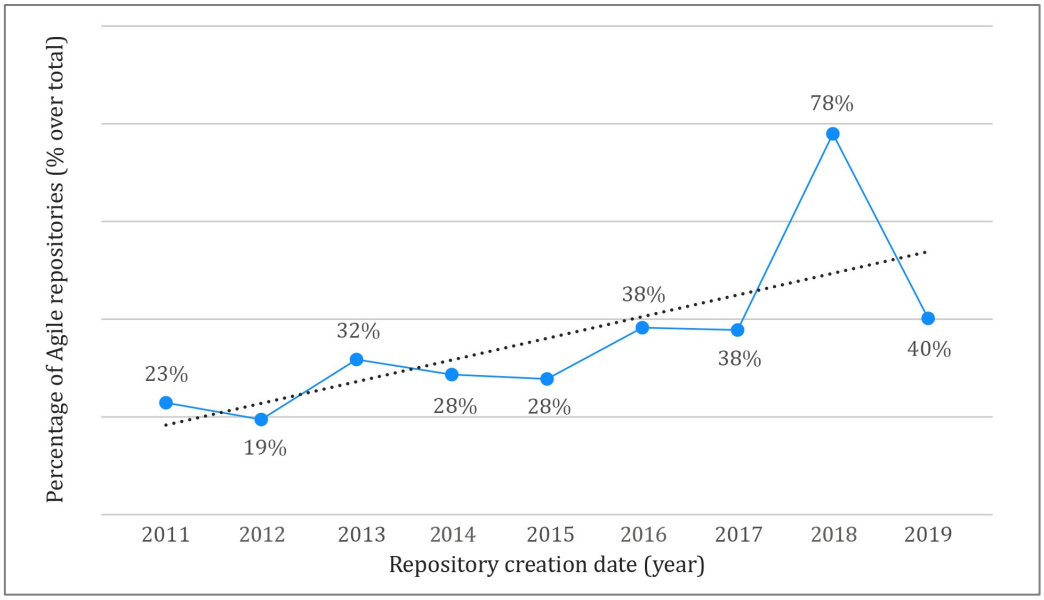

Figure 8. Project methodology evolution over time (percentage of Agile projects according to AgileScore). A linear trendline highlights the positive trend in AgileScore, indicating a growing adoption of Agile methodologies over time.

Projects with a high AgileScore (≥8), on average, exhibit a higher number of watchers among the developer community compared to projects with a very low AgileScore (≤3), which correspond to non-Agile projects (Table 4).

Table 4. Average number of watchers related to non-Agile and Agile projects.

AgileScore	Average Num. of Watchers (±std)
High, meaning Agile projects (8 to 10)	48.75 ± 2.54
Low, meaning non-Agile projects (1 to 3)	37.33 ± 5.53

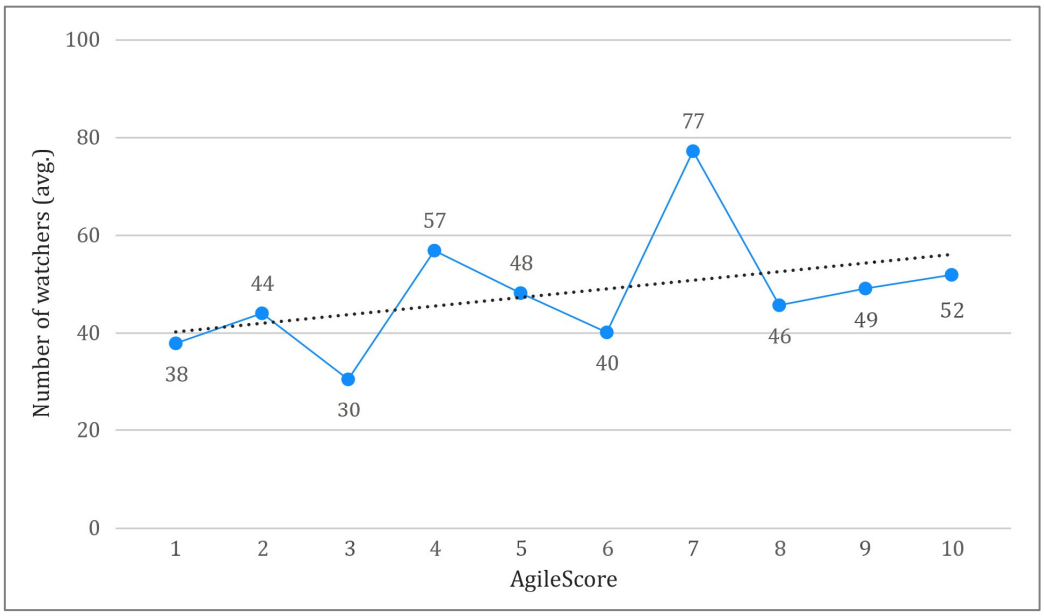

Figure 9. Number of watchers related to the AgileScore index. A linear trendline is included to illustrate the increasing trend in the number of watchers as the AgileScore increases.

5. Discussion

Related to the main research focus regarding which methodologies are more relevant in terms of popularity and community engagement, the application of the AgileScore index on repositories in this study reveals a growing trend in the adoption of Agile methodologies on GitHub, with an average annual increase of 16% (±0.47) over the analyzed period (Figure 8). This trend aligns with recent studies in Kurhmann et al. [36] showing a shift in software development practices toward more flexible and adaptive approaches. This growth may be linked to the need for organizations to respond rapidly to changes in customer requirements and a competitive market environment, thus promoting methodologies that encourage collaboration and incremental value delivery. As also indicated in Kurhmann et al. [36], our study suggests that the spectrum of applied development models is not binary (100% Agile vs. 100% traditional), but rather shows a range or level of agility in the projects, according to the attributes used for identifying the development model (Figures 6 and 7). Considering studies that analyze hybrid models [9–11], this suggests that levels close to the agility thresholds used in our study could be used to identify hybrid models in the dataset.

The AgileScore index also proves useful for analyzing the characteristics of Agile projects in relation to business and community behavior. It enhances and builds upon previous studies aimed at optimizing the Agile development process in open-source environments, such as the work by Peng et al. [38]. The AgileScore index was applied to examine the dynamics of developer and collaborator communities in open-source Agile projects (Figure 9). Results indicate that Agile projects, on average, attract over 10 percentage points more observers than non-Agile projects during the same period (Table 4). This correlation between project popularity and community engagement underscores the importance of visibility and appeal in collaborative environments. This suggests that Agile practices are becoming increasingly popular in collaborative environments, driven by their flexibility, adaptability and capacity to respond to changing customer requirements and competitive pressures.

The proposed model can be expanded to add different research perspectives. By incorporating other metadata attributes, the proposed index can be extended to explore additional relevant areas in project management and software engineering. Following

a previous study on digital innovation [39], the role of the Agile approach in business transformation through technological projects could be better understood by analyzing the types or industry sectors of institutions that most frequently adopt Agile models, as well as organizations implementing Agile methodologies for the first time in their technological projects. For this purpose, the "owner.type" metadata attribute should be incorporated into the analysis model.

Another angle for expanding the model is to examine the range and types of technologies and programming languages employed, as well as their current trends, incorporating metadata attributes such as "languages," similar to the relationship analysis of watchers for AgileScore in Figure 9 and to average date to release in Figure 6. Additionally, other formal aspects, such as licensing and project visibility—represented by the metadata attributes "license" and "visibility"—could provide further insight into the landscape of open innovation communities, following studies such as [40].

AgileScore can be applied to research related to the broad area of interest of cost estimation for Agile software projects, identifying projects that use Agile methodologies and extracting relevant characteristics such as development effort, product complexity and frequency of changes.

Further applications of this analysis model could focus on operational aspects of project management, such as comparing defect reports and changes in Agile projects to other development approaches on GitHub, complementing studies like those by Siddiq and Santos [24].

Theoretical and Practical Implications

From a theoretical standpoint, this study introduces AgileScore as a conceptual construct to operationalize agility in software development projects. This means turning the abstract concept of 'agility' into something that can be measured, observed and analyzed in a practical and systematic way in software projects. Unlike traditional approaches that rely on self-reported surveys, manual coding, textual analysis of documentation or maturity models [5], AgileScore offers a data-driven, quantifiable method for classifying development models based on observable behavioral indicators extracted from real software project data on GitHub. This contributes to the ongoing theoretical discourse on how agility manifests in real-world software practices [3], reinforcing the idea that Agile adoption exists along a spectrum rather than as a binary distinction [9]. It also opens new avenues for research into hybrid models and the nuanced interpretation of Agile practices across different contexts and teams.

From a practical perspective, the AgileScore index enables the identification and comparison of Agile, traditional and intermediate (hybrid) projects within large-scale datasets [25,26]. This facilitates research in software project management and engineering by providing a scalable method to filter and analyze project types according to their development approach. Although not yet implemented as a management tool, the model can be expanded to support project leaders and analysts in benchmarking their projects against similar cases, estimating effort and costs and understanding patterns of team dynamics and community engagement [7]. Moreover, AgileScore could be applied in industry or academia to build tools that assist in the identification and monitoring of Agile practices across open-source or enterprise environments [20], supporting business and technical innovations [4].

6. Conclusions and Future Work

This study successfully achieves its three objectives by demonstrating the diversity of development process models in public repositories like GitHub, establishing the feasibility of classifying projects using an agility index derived from empirical data and highlighting

the potential to replicate this classification process across other repositories, contingent on data availability and the adaptability of methodologies. The following research conclusions could be drawn:

(a) Feasibility of classifying projects in public repositories with an agility index (AgileScore): This research confirms that it is possible to classify projects in public repositories, particularly on GitHub, using an agility index based on empirical data. By applying data analysis and machine learning models, this study effectively identifies Agile projects and characterizes their adherence to specific Agile practices. The methodology allows for the classification and evaluation of projects in terms of their performance against specific metrics, demonstrating its practical applicability.
(b) Characterizing the situation of software projects in public repositories concerning development process models, focusing on trend and community engagement: This study shows that public repositories, such as GitHub, host a wide range of development models, ranging from traditional approaches to Agile. This diversity reflects the flexibility of development teams in adapting processes to meet project-specific needs, underscoring the dynamic nature of software development practices, and advises following a hybrid approach in project development.
(c) Replicability of the classification process in other types of repositories: This research proposes the AgileScore index methodology be adapted for use in other public repositories, as long as they provide sufficient and relevant metadata. The ability to replicate the process depends on data availability and the repository's specific attributes, but the core approach is flexible enough to be applied to various platforms.

In conclusion, this study presents both a model for identifying Agile projects through the 'AgileScore' index and an innovative application of this index to explore various characteristics of these projects. By providing a reliable tool for classifying Agile projects, this study not only highlights the growing trend of Agile adoption in open-source repositories but also opens new avenues for analyzing project dynamics, community engagement and other relevant project features. These findings contribute to advancing understanding of Agile methodologies in collaborative environments and offer valuable insights for future research and project management practices.

The methodology employed in this research effectively addressed the posed problem; however, the primary limitation of this study lies in the data extraction technology applied during the process, which constrained the depth of the analysis (due to limitations in GitHub API queries) and the type of repository (as defined by the research goals, only public projects were addressed). Additionally, certain attributes often used to characterize project dynamics, such as Issue Tracking and Contributors and Collaboration, were not included in the final dataset. This exclusion reflects a deliberate trade-off in favor of selecting variables that contribute more directly to the semantic interpretation required to identify Agile characteristics, such as delivery cadence, release frequency and sustained development activity. The commit-level metadata available through the GitHub public dataset in BigQuery were, therefore, prioritized, as they provide a scalable and consistent source of activity data across a wide range of repositories.

In future work, we plan to expand this research by using the AgileScore index to understand further the relationship between mixed software development approaches (hybrid models) and projects in the early phases of development, with minimal code generated, or those led by individuals compared to large teams or corporations. This would enhance understanding of work dynamics and the potential advantages or implications of Agile models for startups, very small businesses (VSBs), or micro-businesses when collaborating with multiple and large organizations on the same project with differing methodologies (e.g., waterfall vs. Agile). Additionally, any time-related behavior of Agile

projects may be part of future work, such as the anomaly identified in the analysis over time (Figure 8), which occurred in 2018.

Author Contributions: Conceptualization, C.M.M. and J.S.G.; methodology, C.M.M.; software, J.G.C.; validation, C.M.M., J.S.G. and J.G.C.; formal analysis, J.G.C.; investigation, C.M.M. and J.S.G.; resources, J.G.C.; data curation, J.G.C.; writing—original draft preparation, C.M.M. and J.S.G.; writing—review and editing, C.M.M., J.S.G. and J.G.C.; visualization, J.G.C.; supervision, C.M.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: This study did not involve human subjects or personal data; therefore, Institutional Review Board approval was not required.

Informed Consent Statement: Informed consent was not applicable for this study.

Data Availability Statement: The data presented in this study are available in Kaggle at https://www.kaggle.com/datasets/74b4630417dafcd293a185ee77846820b2637f7ccf931140358bda79d6b52270 (accessed on 3 February 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Nicholls, D.; Dong, H.; Dacre, N.; Baxter, D.; Ceylan, S. Understanding Agile in Project Management. *Assoc. Proj. Manag.* 2022. Available online: https://www.apm.org.uk/resources/find-a-resource/agile-project-management/ (accessed on 5 June 2024).
2. European Commission and Directorate-General for Informatics. The PM2-Agile Guide 3.0.1. 2021. Available online: https://data.europa.eu/doi/10.2799/162784 (accessed on 17 June 2024).
3. Kuhrmann, M.; Tell, P.; Hebig, R.; Klünder, J.; Münch, J.; Linssen, O.; Pfahl, D.; Felderer, M.; Prause, C.R.; MacDonell, S.G.; et al. What Makes Agile Software Development Agile. *IEEE Trans. Softw. Eng.* **2022**, *48*, 3523–3539. [CrossRef]
4. Highsmith, J.; Cockburn, A. Agile software development: The business of innovation. *Computer* **2001**, *34*, 120–122. [CrossRef]
5. Henriques, V.; Tanner, M. A systematic literature review of agile and maturity model research. *Interdiscip. J. Inf. Knowl. Manag.* **2017**, *12*, 53–73. [CrossRef]
6. Ghimire, D.; Charters, S. The Impact of Agile Development Practices on Project Outcomes. *Software* **2022**, *1*, 265–275. [CrossRef]
7. Strode, D.; Dingsøyr, T.; Lindsjorn, Y. A teamwork effectiveness model for agile software development. *Empir. Softw. Eng.* **2022**, *27*, 56. [CrossRef]
8. Alsaadi, B.; Saeedi, K. *Data-Driven Effort Estimation Techniques of Agile User Stories: A Systematic Literature Review*; Springer: Dordrecht, Netherlands, 2022; Volume 55. [CrossRef]
9. Gemino, A.; Reich, B.H.; Agile, P.M.S. Traditional, and Hybrid Approaches to Project Success: Is Hybrid a Poor Second Choice? *Proj. Manag. J.* **2021**, *52*, 161–175. [CrossRef]
10. Špundak, M. Mixed Agile/Traditional Project Management Methodology—Reality or Illusion? *Procedia–Soc. Behav. Sci.* **2014**, *119*, 939–948. [CrossRef]
11. Kirpitsas, I.K.; Pachidis, T.P. Evolution towards Hybrid Software Development Methods and Information Systems Audit Challenges. *Software* **2022**, *1*, 316–363. [CrossRef]
12. Glass, R.L. The State of the Practice of Software Engineering. *IEEE Softw.* **2003**, *20*, 20–21. [CrossRef]
13. github.com. Octoverse: The State of Open Source and Rise of AI in 2023—The GitHub Blog. Available online: https://github.blog/2023-11-08-the-state-of-open-source-and-ai/ (accessed on 29 May 2024).
14. Alrashedy, K.; Binjahlan, A. How do Software Engineering Researchers Use GitHub? An Empirical Study of Artifacts & Impact. In Proceedings of the 2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM), Flagstaff, AZ, USA, 7–8 October 2024; pp. 118–130. [CrossRef]
15. Khan, A.A.; Khan, J.A.; Akbar, M.A.; Zhou, P.; Fahmideh, M. Insights into software development approaches: Mining Q &A repositories. *Empir. Softw. Eng.* **2024**, *29*, 1–38. [CrossRef]
16. Sheoran, J.; Blincoe, K.; Kalliamvakou, E.; Damian, D.; Ell, J. Understanding 'watchers' on GitHub. In Proceedings of the 11th Working Conference on Mining Software Repositories, Hyderabad, India, 31 May–1 June 2014; pp. 336–339. [CrossRef]
17. Zhang, J.; Sun, Y.; Zhou, Y.; Wu, J.; Jiang, H.; Huang, G. Exploring GitHub Topics: Unveiling Their Content and Potential. In Proceedings of the 2024 IEEE International Conference on Software Services Engineering, SSE, Shenzhen, China, 7–13 July 2024; Institute of Electrical and Electronics Engineers Inc.: New York, NY, USA, 2024; pp. 25–35. [CrossRef]

18. Xiong, Y.; Meng, Z.; Shen, B.; Yin, W. Mining developer behavior across git hub and stack overflow. In *SEKE*; Ksi Research Inc.: Pittsburgh, PA, USA, 2017; pp. 578–583. [CrossRef]
19. Dabbish, L.; Stuart, C.; Tsay, J.; Herbsleb, J. Social coding in GitHub: Transparency and collaboration in an open software repository. In Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work, Seattle, WA, USA, 15 February 2012; pp. 1277–1286. [CrossRef]
20. Kim, Y.; Kim, J.; Jeon, H.; Kim, Y.-H.; Song, H.; Kim, B.; Seo, J. Githru: Visual analytics for understanding software development history through git metadata analysis. *IEEE Trans. Vis. Comput. Graph.* **2021**, *27*, 656–666. [CrossRef] [PubMed]
21. Mrvar, G. Leveraging Open-Source Data for Software Cost Estimation: A Predictive Modeling Approach. Master's Thesis, Utrecht University, Utrecht, The Netherlands, 2023.
22. Tawosi, V.; Moussa, R.; Sarro, F. Agile Effort Estimation: Have We Solved the Problem Yet? Insights From a Replication Study. *IEEE Trans. Softw. Eng.* **2023**, *49*, 2677–2697. [CrossRef]
23. Robles, G.; Capiluppi, A.; Gonzalez-Barahona, J.M.; Lundell, B.; Gamalielsson, J. Gamalielsson. Development effort estimation in free/open source software from activity in version control systems. *Empir. Softw. Eng.* **2022**, *27*, 135. [CrossRef]
24. Siddiq, M.L.; Santos, J.C.S. BERT-Based GitHub Issue Report Classification. In Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering, NLBSE, Lisbon, Portugal, 20 April 2022; Institute of Electrical and Electronics Engineers Inc.: New York, NY, USA, 2022; pp. 33–36. [CrossRef]
25. Tawosi, V.; Al-Subaihin, A.; Moussa, R.; Sarro, F. A Versatile Dataset of Agile Open Source Software Projects. In Proceedings of the 19th International Conference on Mining Software Repositories, Pittsburgh, PA, USA, 23–24 May 2022; Association for Computing Machinery: New York, NY, USA, 2022; Volume 1. [CrossRef]
26. Pickerill, P.; Jungen, H.J.; Ochodek, M.; Maćkowiak, M.; Staron, M. PHANTOM: Curating GitHub for engineered software projects using time-series clustering. *Empir. Softw. Eng.* **2020**, *25*, 2897–2929. [CrossRef]
27. Mckay, J.; Marshall, P. The dual imperatives of action research. *Inf. Technol. People* **2001**, *14*, 46–59. [CrossRef]
28. Hevner, A.; March, S.T.; Ram, S.; Jinsoo, P. Design Science in Information Systems Research. *MIS Q.* **2004**, *28*, 75–105. [CrossRef]
29. Creswell, J.W.; Creswell, J.D. *Research Design: Qualitative; Quantitative, and Mixed Methods Approaches*, 6th ed.; Sage Publications: Thousand Oaks, CA, USA, 2017.
30. Mariscal, G.; Marbán, Ó.; Fernández, C. A survey of data mining and knowledge discovery process models and methodologies. *Knowl. Eng. Rev.* **2010**, *25*, 137–166. [CrossRef]
31. Chapman, P. CRISP-DM 1.0 Step-by-step data mining guide. *SPSS Inc* **2000**, *78*, 1–78. Available online: http://www.crisp-dm.org/CRISPWP-0800.pdf (accessed on 8 July 2024).
32. Wirth, R.; Hipp, J. CRISP-DM: Towards a standard process model for data mining. In Proceedings of the Fourth International Conference on the Practical Application of Knowledge Discovery and Data Mining, Manchester, UK, 11–13 April 2000; pp. 29–39. Available online: https://www.researchgate.net/publication/239585378_CRISP-DM_Towards_a_standard_process_model_for_data_mining (accessed on 8 July 2024).
33. Schröer, C.; Kruse, F.; Gómez, J.M. A systematic literature review on applying CRISP-DM process model. *Procedia Comput. Sci.* **2021**, *181*, 526–534. [CrossRef]
34. Bruce, P. The Open Source Definition. *Open Source Softw.* **2004**, *1*, 321–322. [CrossRef]
35. Kalliamvakou, E.; Singer, L.; Gousios, G.; German, D.M.; Blincoe, K.; Damian, D. The promises and perils of mining GitHub. In Proceedings of the 11th Working Conference on Mining Software Repositories, MSR 2014, New York, NY, USA, 14 April 2014; Association for Computing Machinery, Inc.: New York, NY, USA, 2014; pp. 92–101. [CrossRef]
36. Kuhrmann, M.; Diebold, P.; Münch, J.; Tell, P.; Garousi, V.; Felderer, M.; Trektere, K.; McCaffery, F.; Linssen, O.; Hanser, E.; et al. Hybrid software and system development in practice: Waterfall, scrum, and beyond. In Proceedings of the 2017 International Conference on Software and System Process, Paris, France, 5–7 July 2017; pp. 30–39. [CrossRef]
37. Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
38. Peng, S.; Kalliamvakou, E.; Cihon, P.; Demirer, M. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv* **2023**, arXiv:2302.06590. Available online: http://arxiv.org/abs/2302.06590 (accessed on 24 May 2024).
39. Appio, F.P.; Frattini, F.; Petruzzelli, A.M.; Neirotti, P. Digital Transformation and Innovation Management: A Synthesis of Existing Research and an Agenda for Future Studies. *J. Prod. Innov. Manag.* **2021**, *38*, 4–20. [CrossRef]
40. Srisathan, W.A.; Ketkaew, C.; Jitjak, W.; Ngiwphrom, S.; Naruetharadhol, P. Open innovation as a strategy for collaboration-based business model innovation: The moderating effect among multigenerational entrepreneurs. *PLoS ONE* **2022**, *17*, e0265025. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.